

Lab-16.3 Assignment

Name: D. Naveen

Roll No: 2403A53L01

Batch: 46

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates). **Generate queries:**
- List all appointments for a specific doctor.
- Retrieve patient history by patient ID.
- Count total patients treated by each doctor. **Expected Output:**
- Normalized schema and SQL queries with joins.

The screenshot displays a database management interface with a SQL editor on the left and a results pane on the right. The SQL editor contains the following code:

```
-- Create Tables
-- CREATE TABLES
-- CREATE TABLE Doctors (
--     DoctorID INT PRIMARY KEY,
--     Name VARCHAR(100),
--     Specialization VARCHAR(100)
-- );
-- CREATE TABLE Patients (
--     PatientID INT PRIMARY KEY,
--     Name VARCHAR(100),
--     Age INT,
--     Gender VARCHAR(10)
-- );
-- CREATE TABLE Appointments (
--     AppointmentID INT PRIMARY KEY,
--     DoctorID INT,
--     PatientID INT,
--     AppointmentDate DATE,
--     FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID),
--     FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
-- );
-- Insert Data
-- INSERT INTO Doctors VALUES
-- (101, 'Dr. Ravi', 'Cardiology'),
-- (102, 'Dr. Priya', 'Neurology');
-- INSERT INTO Patients VALUES
-- (1, 'Arjun', 25, 'Male'),
-- (2, 'Divya', 30, 'Female');
-- INSERT INTO Appointments VALUES
-- (1, 101, 1, '2024-03-10'),
-- (2, 102, 2, '2024-03-11'),
-- (3, 101, 2, '2024-03-12');
-- Output Tables
-- SELECT * FROM Doctors;
-- SELECT * FROM Patients;
-- SELECT * FROM Appointments;
-- Required Queries
-- 1. Appointments for a specific doctor
-- SELECT * FROM Appointments
-- WHERE DoctorID = 101;
-- 2. Patient History
-- SELECT p.Name, a.AppointmentDate, d.Name AS DoctorName
-- FROM Appointments a
-- JOIN Patients p ON a.PatientID = p.PatientID
-- JOIN Doctors d ON a.DoctorID = d.DoctorID
-- WHERE p.PatientID = 2;
-- 3. Count patients per doctor
-- SELECT d.Name, COUNT(a.PatientID) AS TotalPatients
-- FROM Doctors d
-- LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
-- GROUP BY d.Name;
```

The results pane shows the output of the queries:

- Doctors Table:**

DoctorID	Name	Specialization
101	Dr. Ravi	Cardiology
102	Dr. Priya	Neurology

- Patients Table:**

PatientID	Name	Age	Gender
1	Arjun	25	Male
2	Divya	30	Female

- Appointments Table:**

AppointmentID	DoctorID	PatientID	AppointmentDate
1	101	1	2024-03-10
2	102	2	2024-03-11
3	101	2	2024-03-12

- Query 1: Appointments for a specific doctor (DoctorID = 101)**

Name	AppointmentDate	DoctorName
Divya	2024-03-12	Dr. Ravi

- Query 2: Patient History (PatientID = 2)**

Name	AppointmentDate	DoctorName
Divya	2024-03-11	Dr. Priya
Divya	2024-03-12	Dr. Ravi

- Query 3: Count patients per doctor**

Name	TotalPatients
Dr. Priya	1
Dr. Ravi	2

Execution Result Explanation

- The database tables Doctors, Patients, and Appointments were successfully created.
- Sample data was inserted into all tables.

- The output shows:
 - Doctors Table with doctor details
 - Patients Table with patient records
 - Appointments Table showing relationships between doctors and patients
- SQL queries were executed successfully:
 - Retrieved appointments for a specific doctor
 - Displayed patient history using JOIN
 - Counted total patients treated by each doctor using aggregation

Task 2 – Real-Time Application: Online Shopping Database Scenario:

Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
 - Retrieve all orders by a user.
 - Find the most purchased product.
 - Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

The screenshot displays a SQL IDE with a script editor on the left and a query runner on the right. The script editor contains the following SQL code:

```

-- DROP TABLES (optional reset)
--
-- CREATE TABLES
--
CREATE TABLE Users (
  UserID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Products (
  ProductID INT PRIMARY KEY,
  ProductName VARCHAR(100) NOT NULL,
  Price DECIMAL(10,2) CHECK (Price > 0)
);

CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  UserID INT,
  OrderDate DATE,
  FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

CREATE TABLE OrderDetails (
  OrderDetailID INT PRIMARY KEY,
  OrderID INT,
  ProductID INT,
  Quantity INT CHECK (Quantity > 0),
  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

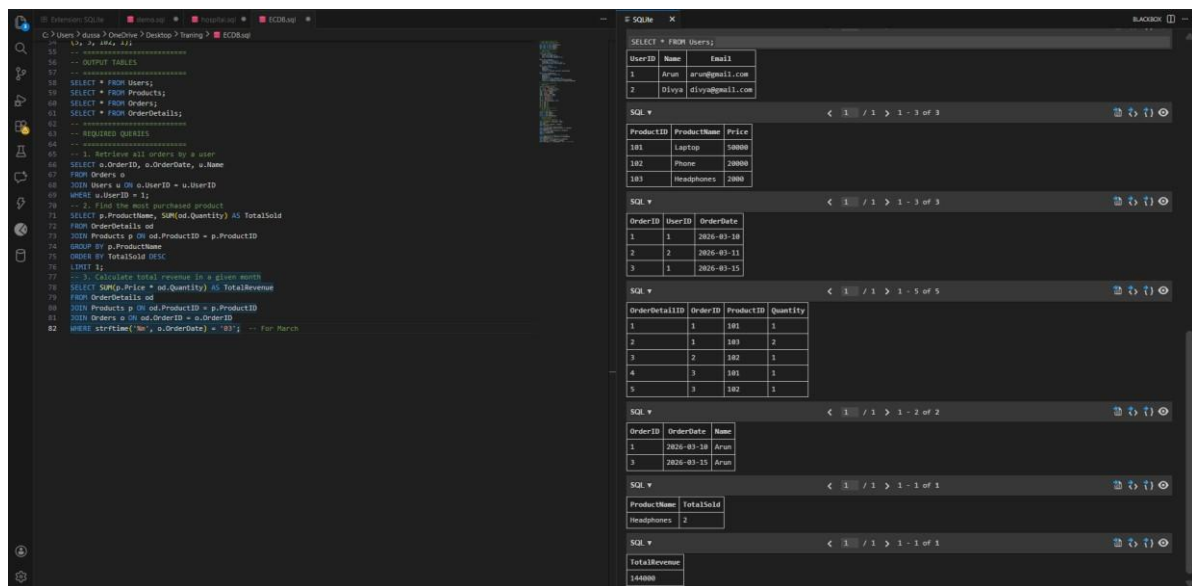
-- INSERT DATA
--
INSERT INTO Users VALUES
(1, 'Arun', 'arun@gmail.com'),
(2, 'Divya', 'divya@gmail.com');

INSERT INTO Products VALUES
(101, 'Laptop', 50000),
(102, 'Phone', 20000),
(103, 'Headphones', 2000);

INSERT INTO Orders VALUES
(1, 1, '2020-03-10'),
(2, 2, '2020-03-11'),
(3, 1, '2020-03-12');

INSERT INTO OrderDetails VALUES
(1, 1, 101, 1),
(2, 1, 102, 2),
(3, 2, 102, 1),
(4, 3, 101, 1),
(5, 3, 102, 1);
  
```

The query runner on the right shows the execution of several queries, including the creation of tables, insertion of data, and retrieval of data. The results are displayed in a table format.



Normalization & Improvements

- Data is split into **Users, Products, Orders, OrderDetails** (3NF).
- Avoids redundancy (no repeated product/user data).
- OrderDetails handles **many-to-many relationship**.

Explanation:

The E-commerce database is designed using four tables: Users, Products, Orders, and Order Details.

Primary keys and foreign keys are used to maintain relationships and ensure data integrity.

The schema is normalized to eliminate redundancy and organize data efficiently. SQL queries with JOIN and aggregation functions are used to retrieve user orders, identify the most purchased product, and calculate total revenue.

Query performance is improved using indexing and optimized joins instead of subqueries.

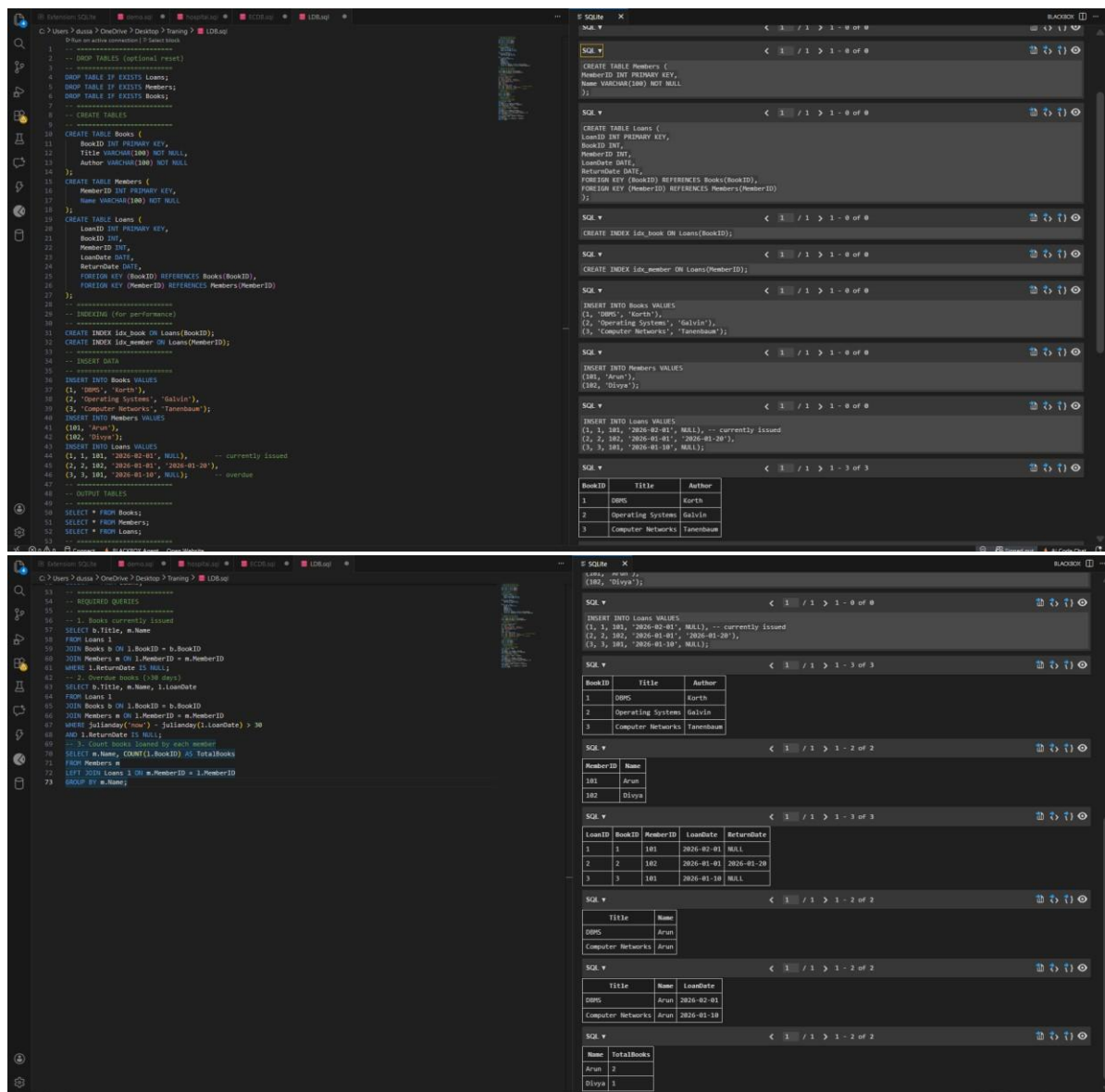
Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- **Generate queries:**
- Retrieve all books currently issued.
- Find overdue books (loan date > 30 days).
- Count number of books loaned by each member. **Expected Output:**
- Schema + SQL queries demonstrating joins and conditions.



Explanation:

The Library Management Database is designed using Books, Members, and Loans tables.

Primary and foreign keys maintain relationships and ensure data integrity.

Indexing is applied on Loan table columns to improve query performance.

SQL queries using JOIN and conditions retrieve issued books, overdue books, and count loans per member.

The schema is normalized to avoid redundancy and improve efficiency.

Task 4: Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

Original query

```
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');
```

Optimized query

```
SELECT b.*
```

```
FROM Books b
```

```
JOIN Authors a ON b.author_id =
```

```
a.author_id WHERE a.country = 'UK';
```

Expected Output:

- Both queries return the same result: all books written by UK authors.
- Optimized query performs faster for larger datasets

The screenshot shows a database IDE with two panels. The left panel displays the SQL script, and the right panel shows the execution results.

SQL Script (Left Panel):

```
-- DROP TABLES (reset)
-- DROP TABLE IF EXISTS Books;
-- DROP TABLE IF EXISTS Authors;

-- CREATE TABLES
-- =====
CREATE TABLE Authors (
  author_id INT PRIMARY KEY,
  name VARCHAR(100),
  country VARCHAR(50)
);

CREATE TABLE Books (
  BookID INT PRIMARY KEY,
  Title VARCHAR(100),
  author_id INT,
  FOREIGN KEY (author_id) REFERENCES Authors(author_id)
);

-- INSERT DATA
-- =====
INSERT INTO Authors VALUES
(201, 'John Smith', 'UK'),
(202, 'David Miller', 'UK'),
(203, 'Ravi Kumar', 'India');

INSERT INTO Books VALUES
(1, 'Database Systems', 201),
(2, 'Operating Systems', 203),
(3, 'Data Structures', 201),
(4, 'AI Basics', 202),
(5, 'Computer Networks', 202);

-- ORIGINAL QUERY
SELECT *
FROM Books
WHERE author_id IN (
  SELECT author_id
  FROM Authors
  WHERE country = 'UK'
);

-- OPTIMIZED QUERY
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Execution Results (Right Panel):

SQL #1: DROP TABLE IF EXISTS Books;

SQL #2: DROP TABLE IF EXISTS Authors;

SQL #3: CREATE TABLE Authors (...)

SQL #4: CREATE TABLE Books (...)

SQL #5: INSERT INTO Authors VALUES (...)

SQL #6: INSERT INTO Books VALUES (...)

SQL #7: ORIGINAL QUERY

BookID	Title	author_id
1	Database Systems	201
3	Data Structures	201
5	Computer Networks	202

SQL #8: OPTIMIZED QUERY

BookID	Title	author_id
1	Database Systems	201
3	Data Structures	201
5	Computer Networks	202

Explanation

The original query uses a subquery to filter books based on authors from the UK, which can be less efficient for large datasets.

The optimized query replaces the subquery with a JOIN, allowing faster data retrieval by directly combining both tables.

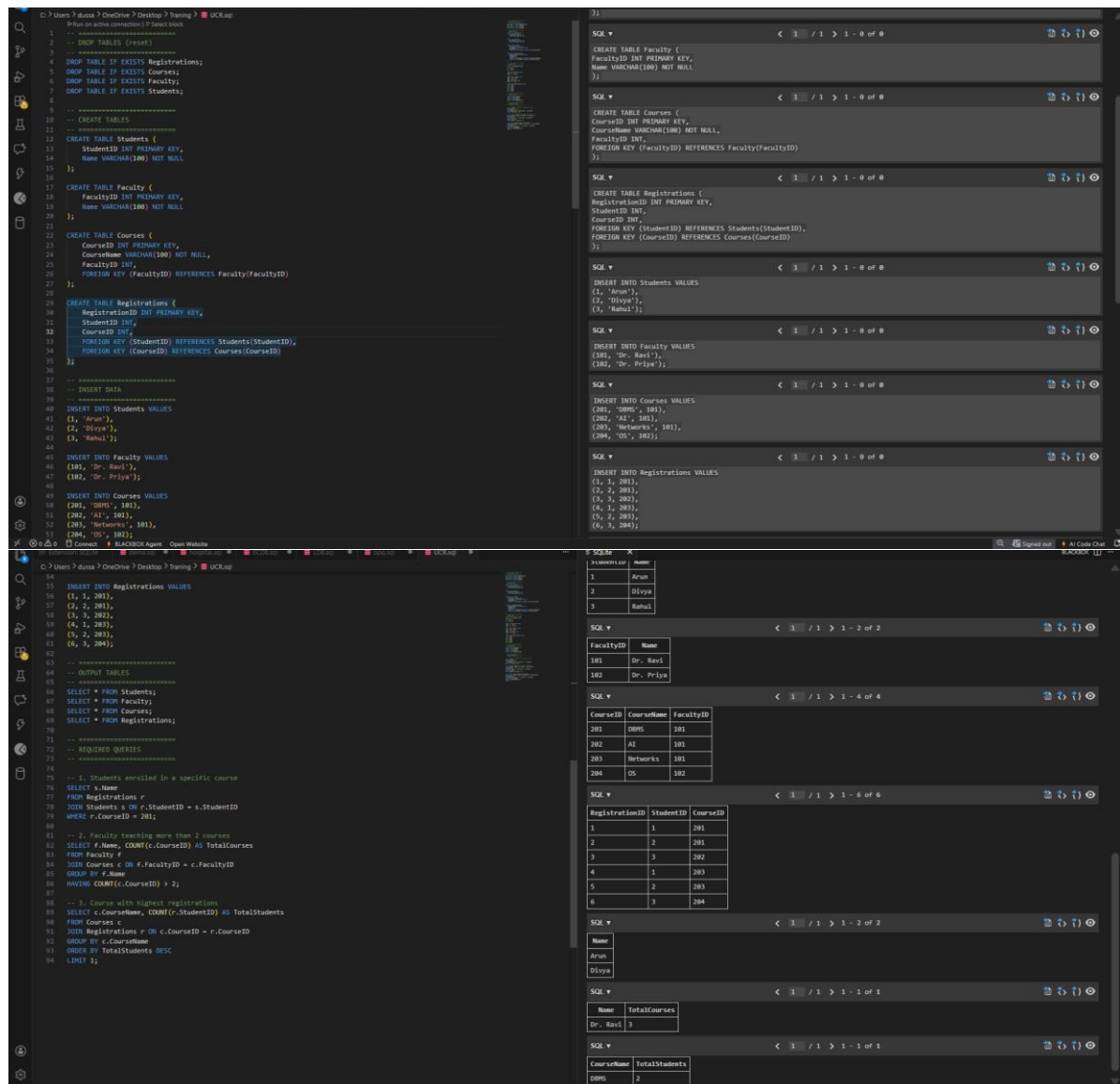
JOIN operations are more efficient as they utilize indexing and reduce repeated scanning of tables.

Both queries return the same result, but the optimized query improves performance and scalability. Thus, query optimization helps in reducing execution time and improving database efficiency.

Scenario:

SR University needs a course registration system for students enrolling in different courses under faculty supervision.

- Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
- Define relationships:
- Each student can register for multiple courses.
- Each course is taught by a faculty member.
- **Write SQL queries to:**
- List all students enrolled in a specific course.
- Find faculty members teaching more than 2 courses.
- Retrieve courses with the highest number of registrations.



Explanation

The University Course Registration system is designed using Students, Faculty, Courses, and Registrations tables.

Relationships are maintained using primary and foreign keys to ensure data integrity.

Each student can enroll in multiple courses, and each course is assigned to a faculty member. SQL queries using JOIN and aggregation are used to retrieve enrolled students, identify faculty teaching multiple courses, and find the most popular course. The schema is normalized to reduce redundancy and improve efficiency.